

Data Structures Deep Dive — Smart Parking System

Why each data structure was chosen, how it works, what happens in the best, average, and worst case — explained so a 6-year-old who has never heard the words "Big O" or "algorithm" can follow along. Every concept has a real-world toy story. Every case has working ASCII diagrams.



Table of Contents

- [What Even IS a Data Structure?](#)
- [DS 1 — Array \(The Parking Lot Grid\)](#)
- [DS 2 — Hash Map \(The Magic Phonebook\)](#)
- [DS 3 — Stack — LIFO \(The Undo Button\)](#)
- [DS 4 — Queue — FIFO \(The Fair Line\)](#)
- [DS 5 — Linked List \(The Treasure Hunt Chain\)](#)
- [How All 5 Work Together](#)
- [Speed Cheat Sheet](#)



What Even IS a Data Structure?

Imagine you have 100 marbles. You need to find the red one.

Option A: Throw all 100 into a bag, shake, dig, hope.

Option B: Sort them into labeled cups by colour.

A **data structure** is just *how you organise your stuff* so finding, adding, or removing things is easy and fast. Different problems need different organisers — just like you'd use a backpack for school, a fridge for food, and a wallet for money. You wouldn't store your sandwiches in your wallet.

This project uses **5 different data structures**, each chosen for a specific job. Let's meet them one by one.



DS 1 — Array (The Parking Lot Grid)

The Toy Story Explanation

Picture a row of 10 numbered toy boxes at school: Box 1, Box 2, Box 3... Box 10. Every box has a name tag saying whether it's EMPTY or TAKEN. When a car wants a spot, you look at Box 1 first, then Box 2, and so on — you find the first empty one.

That's exactly an **array** — a row of boxes with numbers on them.

Why We Used It Here

The parking lot has exactly **10 fixed slots**. Slot 1–5 are for bikes, Slot 6–10 are for cars. The number never changes. Arrays are perfect when you know the exact size upfront because every slot gets a permanent address (like a house number on a street).

```
1  CODE:  ParkingSlot lot[MAX_SLOTS];    // MAX_SLOTS = 10
2          lot[0] = Slot 1 (BIKE)
3          lot[1] = Slot 2 (BIKE)
4          ...
5          lot[9] = Slot 10 (CAR)
```

What Each Slot Holds

```
1  +-----+
2  | lot[5] (= Slot 6, CAR slot) |
3  | +-----+-----+-----+ |
4  | | slot_id = 6 | status = OCCUPIED | slot_type = CAR | |
5  | +-----+-----+-----+ |
6  | | plate = "DL4CAB5678" | owner = "Priya" | |
7  | +-----+-----+-----+ |
8  | | entry_time = 1746703000 | |
9  | +-----+-----+-----+ |
10 +-----+
```

The Find-a-Free-Slot Operation

When a CAR arrives, the code calls `find_free_slot(TYPE_CAR)`. It starts at index 0 and checks each slot:

```
1  find_free_slot(CAR):
2
3  lot[0] BIKE slot -> skip (wrong type)
4  lot[1] BIKE slot -> skip (wrong type)
5  lot[2] BIKE slot -> skip (wrong type)
6  lot[3] BIKE slot -> skip (wrong type)
7  lot[4] BIKE slot -> skip (wrong type)
```

```

8  lot[5] CAR slot -> OCCUPIED -> skip
9  lot[6] CAR slot -> OCCUPIED -> skip
10 lot[7] CAR slot -> FREE! -> return index 7

```

Example 1 — BEST CASE

Situation: A CAR arrives and Slot 6 (the very first CAR slot, index 5) is free.

```

1  BEFORE :
2  +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
3  | S1    | S2    | S3    | S4    | S5    | S6    | S7    | S8    | S9    | S10   |
4  | BIKE  | BIKE  | BIKE  | BIKE  | BIKE  | CAR   | CAR   | CAR   | CAR   | CAR   |
5  | FREE  | FREE  | FREE  | FREE  | FREE  | FREE  | OCC   | OCC   | OCC   | OCC   |
6  +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
7                                     ^
8                                     start checking CAR slots here
9
10 Steps taken: 1 (found it immediately at the first CAR slot)

```

Plain English: You walk up to the first CAR box, peek inside, it's empty, done. One peek and you're finished. It's like reaching into your bag for your water bottle and it's right on top.

Example 2 — AVERAGE CASE

Situation: A CAR arrives and the first 3 CAR slots are occupied, slot 9 is free.

```

1  BEFORE :
2  +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
3  | S1    | S2    | S3    | S4    | S5    | S6    | S7    | S8    | S9    | S10   |
4  | BIKE  | BIKE  | BIKE  | BIKE  | BIKE  | CAR   | CAR   | CAR   | CAR   | CAR   |
5  | FREE  | FREE  | FREE  | FREE  | FREE  | OCC   | OCC   | OCC   | FREE  | OCC   |
6  +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
7                                     X      X      X      ^
8                                     skip  skip  skip  FOUND! (4th check)

```

Plain English: You peek into 4 boxes before finding an empty one. Not instant, not terrible. Like checking your bag pockets one by one until you find your keys.

Example 3 — WORST CASE

Situation: A BIKE arrives but ALL 5 bike slots are full.

```
1  BEFORE :
2  +-----+-----+-----+-----+-----+-----+-----+-----+
3  | S1    | S2    | S3    | S4    | S5    | S6    | S7    | S8    | S9    | S10   |
4  | BIKE  | BIKE  | BIKE  | BIKE  | BIKE  | CAR   | CAR   | CAR   | CAR   | CAR   |
5  | OCC   | OCC   | OCC   | OCC   | OCC   | OCC   | OCC   | OCC   | OCC   | OCC   |
6  +-----+-----+-----+-----+-----+-----+-----+-----+
7  X       X       X       X       X
8  check  check  check  check  check -> reach end -> NO SLOT FOUND -> join
   queue
9                                     (5 checks, nothing found)
```

Plain English: You open every single bike box, all 5 are taken. You had to check all of them before knowing there's no room. This is the slowest it can get — like reading a full dictionary to find out a word isn't in it.

What happens next: The bike joins the waiting **Queue** (DS 4 below) — like taking a token at a crowded DMV.

DS 2 — Hash Map (The Magic Phonebook)

The Toy Story Explanation

Imagine a school with 20 cubbyholes on a wall (numbered 0–19). Every kid's name gets run through a "name machine" that spits out a cubbyhole number. So "RAHUL" goes in, the machine says "7", and Rahul's stuff goes in cubbyhole 7. Next time you need Rahul's stuff, run "RAHUL" through the same machine — it says "7" — you go straight to cubbyhole 7. No searching, no guessing.

That machine is called a **hash function**. The wall of cubbyholes is the **hash map**.

Why We Used It Here

When a car leaves, you need to find *which slot it's parked in, given only its licence plate*. Without a hash map, you'd scan all 10 slots every time. With the hash map, you just run the plate through the hash function and jump straight to its cubbyhole in the `buckets[]` array of size 20 (`HASH_SIZE = 20`).

```
1  CODE:  HashMap  plate_map;           // 20 buckets
2          buckets[0]  -> NULL
3          buckets[1]  -> [MH12AB1234 -> Slot 1] -> NULL
4          ...
5          buckets[7]  -> [DL4CAB5678 -> Slot 6] -> [GJ01HH7890 -> Slot 3] ->
   NULL
6          ...
```

The Hash Function (djb2)

C *C*

```
1 unsigned long hash = 5381;
2 for each character c in plate:
3     hash = hash * 33 + c
4
5 final_bucket = hash % 20 // keep it within 0-19
```

Plain English: It's like a very specific recipe. You feed in each letter of the plate one at a time, mix it with the current total, and at the end you get a bucket number between 0 and 19. Same plate ALWAYS gives the same bucket.

What a Collision Is

Sometimes two different plates get the same bucket number. That's called a **collision**. Like two kids whose names both map to cubbyhole 7 — so you chain them together like a mini-list inside the same cubbyhole.

```
1 bucket[7]: [MH12AB1234->Slot1] --> [GJ01HH7890->Slot3] --> NULL
2                                     ^
3                                     collision! two plates
4                                     hashed to bucket 7
```



Example 1 — BEST CASE (no collision)

Situation: Search for plate `MH12AB1234`. It hashes to bucket 3. Bucket 3 has exactly one entry and it's the one we want.

```
1 hashmap_search("MH12AB1234"):
2
3 Step 1: hash("MH12AB1234") % 20 = 3
4 Step 2: Go to bucket[3]
5
6 bucket[3]: [MH12AB1234 -> Slot 1] -> NULL
7             ^
8             MATCH! Return Slot 1
9
10 Total nodes checked: 1
```

Plain English: Run the name through the machine, open that one cubbyhole, first thing inside is exactly what you wanted. Done in one move. Like unlocking your own locker — no searching.



Example 2 — AVERAGE CASE (small chain)

Situation: Three plates all hashed to bucket 7. You're looking for the second one.

```
1  hashmap_search("GJ01HH7890"):
2
3  Step 1: hash("GJ01HH7890") % 20 = 7
4  Step 2: Go to bucket[7]
5
6  bucket[7]: [MH12AB1234 -> Slot 1] --> [GJ01HH7890 -> Slot 3] -->
              [WB20IJ4567 -> Slot 5] --> NULL
7
8              |                               |
9              no match,                       MATCH!
              move to next                     Return Slot 3
10
11 Total nodes checked: 2
```

Plain English: You open cubbyhole 7, the first thing is someone else's stuff. You look at the note on it that says "check the next one" — you do — that's the one. Two checks total.



Example 3 — WORST CASE (all in one bucket)

Situation: Pathological input — somehow all 10 vehicles hash to bucket 12. You're looking for the last one.

```
1  hashmap_search("LAST_PLATE"):
2
3  Step 1: hash("LAST_PLATE") % 20 = 12
4  Step 2: Go to bucket[12]
5
6  bucket[12]: [plate1->S1] -> [plate2->S2] -> [plate3->S3] -> ... ->
              [LAST_PLATE->S10] -> NULL
7
8              check           check           check           ...
9  MATCH!
10
11 Total nodes checked: 10 (scanned the entire chain)
```

Plain English: Every single kid's name somehow got assigned to the same cubbyhole. You have to dig through all 10 items piled on top of each other. This almost never happens with a good hash

function — which is why the code uses the battle-tested djb2 algorithm. But mathematically it *could* happen.

DS 3 — Stack — LIFO (The Undo Button)

The Toy Story Explanation

Picture a stack of plates in a cafeteria. You always put a new plate ON TOP and always take from the TOP. You can never take from the bottom without removing everything above it first.

L.I.F.O. = Last In, First Out. The most recently added thing is the first one you get back.

Your phone's UNDO button works exactly like this. Every action you take is pushed onto the stack. When you hit Undo, it pops the most recent action off the top.

Why We Used It Here

Every time a car parks or leaves, we create a **log entry** and push it onto the stack. When you view the Entry/Exit Log (menu option 6), it shows the most recent event first — exactly what makes sense for a log/history. The stack holds up to `STACK_CAPACITY = 50` entries.

```
1 CODE:   LogStack log_stack;
2         log_stack.top    = index of most recent entry
3         log_stack.entries = array of LogEntry[50]
```

Visual — How Stack Grows

	START: CAR2 LEAVES:	AFTER CAR1 PARKS:	AFTER CAR2 PARKS:	AFTER (pop from top)
1	top = -1	top = 0	top = 1	top =
2				
3				
4				
5		ENTRY CAR1	ENTRY CAR2	
6			ENTRY CAR1	
7				
8	+-----+	+-----+	+-----+	+-----+
9	(empty)			
10	ENTRY CAR1			
11				
12				

Note: When we pop on a removal, we also write a new EXIT entry. The stack grows by 1 on park, grows by 1 on remove (the EXIT log), and displaying is read-only (no pop).



Example 1 — BEST CASE (push when stack is empty)

Situation: Car parks for the first time. Stack is empty (`top = -1`).

```
1  BEFORE push:                                AFTER push (log: ENTRY, Slot 1,
    MH12AB1234):
2
3  top = -1                                    top = 0
4  +-----+                                +-----+
5  | entries[49]  [empty]  |                | entries[49]  [empty]  |
6  | entries[48]  [empty]  |                | entries[48]  [empty]  |
7  | ...          |                | ...          |
8  | entries[1]   [empty]  |                | entries[1]   [empty]  |
9  | entries[0]   [empty]  |                | entries[0]   ENTRY,S1, | <- top
10 +-----+                                |          MH12AB1234 |
11                                     +-----+
12
13 Steps: 1 (just write to entries[top+1] and increment top)
```

Plain English: The stack is empty. You place the first plate down. One move, instant, can't be easier.



Example 2 — AVERAGE CASE (push mid-way through)

Situation: 25 events already logged. A new event comes in.

```
1  BEFORE:                                AFTER:
2  top = 24                                top = 25
3
4  entries[25]  [empty]                    entries[25]  EXIT, Slot6, HR26KL8901
   <- top
5  entries[24]  ENTRY,S3,[...] <-top  entries[24]  ENTRY, S3, [...]
6  ...
7  entries[0]   ENTRY,S1,[...]          entries[0]   ENTRY, S1, [...]
8
9  Steps: 1 (always just one step regardless of how full it is)
```

Plain English: No matter how many plates are already in the stack, adding a new one is always exactly one move. You always know exactly where the top is.


```

4  +-----+      +-----+      +-----+
5  | Harpreet |---->| Suresh   |---->| Ravi     |
6  | (arrived |      | (arrived |      | (arrived |
7  | first)   |      | second)  |      | third)   |
8  +-----+      +-----+      +-----+
9
10 Slot frees up -> Harpreet dequeues from FRONT, gets the slot
11 Harpreet leaves queue:
12
13 FRONT                                REAR
14 |                                    |
15 v                                    v
16 +-----+      +-----+
17 | Suresh   |---->| Ravi     |
18 +-----+      +-----+
19
20 New vehicle arrives -> joins at REAR:
21
22 FRONT                                REAR
23 |                                    |
24 v                                    v
25 +-----+      +-----+      +-----+
26 | Suresh   |---->| Ravi     |---->| Neha     |
27 +-----+      +-----+      +-----+

```



Example 1 — BEST CASE (enqueue to empty queue)

Situation: Lot is full. First vehicle (Harpreet, BIKE) joins the empty waiting list.

```

1  BEFORE:                                AFTER:
2  front = NULL                            front ----> [Harpreet, BIKE]
3  rear  = NULL                            rear  ----> [Harpreet, BIKE]
4  count = 0                              count = 1
5
6                                          Note: front and rear both point to
7                                          the SAME node when only 1 is waiting.
8
9  Steps: 1 (create node, set both front and rear to it)

```

Plain English: Nobody was waiting. Harpreet is first. You write her name on a piece of paper and that paper is both the "front of line" AND the "back of line" at the same time, because she's alone.



Example 2 — AVERAGE CASE (dequeue from middle-sized queue)

Situation: 3 people waiting. A slot frees up. The front person (Harpreet) gets assigned.

```
1  BEFORE:
2  front -> [Harpreet] --> [Suresh] --> [Ravi] <- rear
3  count = 3
4
5  Step 1: Read front node (Harpreet, BIKE)
6  Step 2: Move front pointer to next node (Suresh)
7  Step 3: Free the old front node (Harpreet)
8  Step 4: Assign Harpreet to the freed slot
9
10 AFTER:
11 front -> [Suresh] --> [Ravi] <- rear
12 count = 2
13
14 Steps: 1 (always just reads and moves the front pointer)
```

Plain English: The first person in line steps forward, takes their ice cream, and leaves. The second person is now first. One move, done.



Example 3 — WORST CASE (queue is completely full)

Situation: 20 vehicles are already waiting (`QUEUE_CAPACITY = 20`). A 21st vehicle tries to join.

```
1  BEFORE:
2  front -> [V1] --> [V2] --> ... --> [V20] <- rear
3  count = 20
4
5  New vehicle arrives (V21):
6  queue_is_full() returns TRUE (count >= QUEUE_CAPACITY)
7  enqueue() blocked
8
9  System message: "Waiting list is also full. Cannot accommodate this
   vehicle."
10
11 AFTER: no change
12 count = 20
```

Plain English: The ice cream line has 20 kids and the teacher said maximum 20. Kid 21 shows up and is told to come back later. The line doesn't grow past its limit.

Design note: In a real parking system you'd tell the driver to try again later or direct them to a different lot. Here the system simply rejects them with a message.

DS 5 — Linked List (The Treasure Hunt Chain)

The Toy Story Explanation

Imagine a treasure hunt. Clue #1 is under your pillow. When you find it, it says "next clue is in the fridge." The fridge clue says "next clue is in the garden." Each clue (node) holds the treasure AND a direction to the next clue.

That's a **Linked List**. Each piece of data carries a pointer (arrow) saying where the next piece lives in memory. They don't need to be next to each other physically — they just know who comes next.

Why We Used It TWICE Here

Linked lists appear in **two places** in this project:

Place 1 — Hash Map Chaining: When two plates hash to the same bucket, they form a mini linked list inside that bucket (each node points to the next collision victim).

Place 2 — The Waiting Queue Nodes: Each vehicle in the queue is a `WaitingNode` containing the vehicle data + a `next` pointer to the next waiting vehicle.

C*C*

```
1 // Hash map node
2 struct HashNode {
3     char    plate[15];
4     int     slot_id;
5     HashNode* next;    // <- pointer to next node in same bucket
6 };
7
8 // Queue node
9 struct WaitingNode {
10     Vehicle    vehicle;
11     WaitingNode* next;    // <- pointer to next vehicle in queue
12 };
```

The Key Difference from an Array

```
1 ARRAY (fixed houses on a street):
2 +---+---+---+---+---+
3 | S1 | S2 | S3 | S4 | S5 |
```

```

4  +---+---+---+---+---+
5  [0] [1] [2] [3] [4]    <- addresses fixed, never move
6
7  LINKED LIST (treasure hunt):
8  [NodeA] ---next---> [NodeB] ---next---> [NodeC] ---next---> NULL
9      |                   |                   |
10     anywhere            anywhere            anywhere
11     in memory           in memory           in memory

```

Array: All spots exist upfront, empty or occupied. Fixed size. Direct jump to any index.

Linked List: Nodes are created on demand and chained together. Grows and shrinks freely. But to get to node 5, you must pass through nodes 1, 2, 3, 4 first.



Example 1 — BEST CASE (insert/search at head of chain)

Situation: Plate `MH12AB1234` is inserted and there's nothing in bucket 3 yet.

```

1  BEFORE inserting MH12AB1234:
2  bucket[3] -> NULL
3
4  Step 1: Create new HashNode (plate="MH12AB1234", slot_id=1, next=NULL)
5  Step 2: Set bucket[3] = new node
6
7  AFTER:
8  bucket[3] -> [MH12AB1234 | Slot 1 | next=NULL]
9
10 Later, searching for MH12AB1234:
11 Go to bucket[3], first node IS the match -> return Slot 1 immediately
12
13 Nodes traversed: 1

```

Plain English: First clue is exactly what you were looking for. You didn't even need to follow the chain. One look and done.



Example 2 — AVERAGE CASE (insert as new head, search in mid-chain)

Situation: Insert a new plate that collides with an existing one. The code inserts it at the HEAD (front) of the chain (faster than walking to the tail).

```

1  BEFORE:
2  bucket[7] -> [GJ01HH7890 | Slot 3 | next=NULL]

```

```

3
4   Inserting [MH43CD2345 | Slot 8]:
5   Step 1: new_node->next = bucket[7]      (point new node to old head)
6   Step 2: bucket[7] = new_node           (new node becomes new head)
7
8   AFTER:
9   bucket[7] -> [MH43CD2345 | Slot 8 | next ->] -> [GJ01HH7890 | Slot 3 |
    next=NULL]
10
11  Searching for GJ01HH7890:
12  bucket[7] first node: MH43CD2345 -- not a match, follow next
13  bucket[7] second node: GJ01HH7890 -- MATCH, return Slot 3
14
15  Nodes traversed: 2

```

Plain English: The newest clue is put at the entrance of the treasure trail, so if you're looking for the new thing you find it fast. But the old clue is now one step deeper in the chain.



Example 3 — WORST CASE (long chain traversal, deletion)

Situation: Delete plate `DL4CAB5678` which is buried at the END of a 5-node collision chain in bucket 12.

```

1   bucket[12]:
2   [plate1->S1] -> [plate2->S2] -> [plate3->S3] -> [plate4->S4] ->
    [DL4CAB5678->S6] -> NULL
3       prev=NULL           prev           prev           prev
    TARGET
4
5   Search process:
6   Node 1: plate1      != DL4CAB5678, move on  (track prev = node1)
7   Node 2: plate2      != DL4CAB5678, move on  (track prev = node2)
8   Node 3: plate3      != DL4CAB5678, move on  (track prev = node3)
9   Node 4: plate4      != DL4CAB5678, move on  (track prev = node4)
10  Node 5: DL4CAB5678 == MATCH!
11
12  Deletion:
13  prev(node4)->next = node5->next  (= NULL)
14  free(node5)
15
16  AFTER:
17  [plate1->S1] -> [plate2->S2] -> [plate3->S3] -> [plate4->S4] -> NULL

```

Plain English: You have to follow 5 clues before you find the right one, then carefully re-link the chain to skip it — like removing a link from a paper chain without tearing the rest. Each step is cheap

but you have no shortcut to the end.



How All 5 Work Together

This is the magic moment — all five DS working as one machine for a single operation:

Operation: Vehicle Arrives

```
1  1. ARRAY scan:      find_free_slot() scans lot[] for a free slot
2                      |
3                      v
4  2. HASH MAP check:  hashmap_search(plate) -- is this plate already
                        parked?
5                      If yes -> reject (duplicate)
6                      |
7                      v
8  3a. Slot found:     lot[idx].status = OCCUPIED    (ARRAY write)
9                      hashmap_insert(plate, slot_id) (HASH MAP write)
10                     stack_push(log_entry)          (STACK push)
11                     |
12  3b. No slot found:  queue_enqueue(vehicle)        (QUEUE + LINKED LIST)
```

```
1  FULL FLOW DIAGRAM:
2
3  Vehicle arrives
4      |
5      v
6  +-----+
7  | HASH MAP search |   Is this plate already inside?
8  | O(1) avg lookup |   YES -> "Already parked!" reject
9  +-----+
10     | NO
11     v
12  +-----+
13  | ARRAY scan      |   Is there a free slot of correct type?
14  | scan lot[0..9]  |   NO -> go to queue
15  +-----+
16     | YES
17     |
18     |   +-----+
19     |   | QUEUE enqueue           |
20     |   | LINKED LIST node created |
21     |   | Waits for next free slot |
22     |   +-----+
23  +-----+
```

```

24 | ARRAY write      | Mark slot as OCCUPIED
25 | lot[idx].status  | Save plate, owner, entry_time
26 +-----+
27 |
28 v
29 +-----+
30 | HASH MAP insert | plate -> slot_id stored
31 | O(1) future look |
32 +-----+
33 |
34 v
35 +-----+
36 | STACK push      | ENTRY event logged at top
37 | instant O(1)    |
38 +-----+
39 |
40 v
41 DONE. Diagrams generated.

```

⚡ Speed Cheat Sheet

No jargon, just plain "how many steps does this take?"

Operation	Data Structure Used	Best Steps	Typical Steps	Worst Steps
Find free slot	Array (scan)	1	~half	All slots
Look up plate	Hash Map	1	1–3	All in bucket
Add log entry	Stack (push)	1	1	blocked if full
Read last log	Stack (top)	1	1	1
Add to wait list	Queue + Linked List	1	1	blocked if full
Serve next waiter	Queue (dequeue)	1	1	1
Remove from chain	Linked List	1	~chain/2	Full chain length

The big picture in one sentence:

The Hash Map keeps lookup at near-instant speed. The Array keeps the lot state always visible. The Stack keeps history honest (most recent first). The Queue keeps the waiting list fair. The Linked List lets the Hash Map and Queue grow and shrink without wasting fixed space.

Each structure does one job perfectly so the others don't have to try to do it badly.

Built for learning. Every diagram, every line of C code, every structure here was a deliberate choice.